MovieBookingProfile JavaBean & OTP Property

Practice Problem 4

Concept and approach

Provide both required constructors and chain the name constructor through this(). Every ordinary property follows the JavaBean convention. Store a transformed OTP without any getter, so it cannot be retrieved by callers.

Important implementation notes

The OTP is constrained to a 4-6 digit numeric string before hashing. For production use, verification should compare a salted, purpose-appropriate OTP hash and enforce expiry/attempt limits.

Java solution

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class MovieBookingProfile {
private String name;
private boolean confirmed;
private byte[] otpDigest;

public MovieBookingProfile() { }
public MovieBookingProfile(String name) {
this();
this.name = name;
}

public String getName() { return name; }
public void setName(String name) { this.name = name; }
public boolean isConfirmed() { return confirmed; }
public void setConfirmed(boolean confirmed) { this.confirmed = confirmed; }

public void setOtp(String otp) {
if (otp == null || !otp.matches("\d{4,6}"))
throw new IllegalArgumentException("OTP must be 4-6 digits");
try {
otpDigest = MessageDigest.getInstance("SHA-256")
.digest(otp.getBytes(StandardCharsets.UTF_8));
} catch (NoSuchAlgorithmException e) {
throw new IllegalStateException("SHA-256 is unavailable", e);
}
}
// No getOtp() method and no API exposes otpDigest.
}
```

Sample output

```
Rahul Dev
true
(no observable output from setOtp)
```